

Does It Render Everywhere? A Study of Cross-Environment Compatibility in MLLM-Generated Webpages

Ziyun Guo¹, Jingyu Xiao², Yuqiang Sun³, Yintong Huo¹

¹Singapore Management University, Singapore

²The Chinese University of Hong Kong, Hong Kong SAR, China

³Nanyang Technological University, Singapore

Abstract—Multimodal Large Language Models (MLLMs) have been increasingly adopted to automate webpage generation from visual designs (e.g., screenshots). However, existing evaluations are limited to visual fidelity assessment under a fixed browser-device configuration. Such a setting overlooks the cross-environment rendering compatibility for real-world deployments.

To address this gap, we present the first systematic empirical study of cross-environment compatibility in AI-generated webpages. Specifically, we construct WEBCOMPAT, a dataset of 2,032 annotated instances, comprising webpages generated by 8 representative AI tools, each rendered across 9 browser-and-device combinations. Based on the dataset, we conduct an analysis on the prevalence of compatibility issues, the user-perceptible visual symptoms they manifest, and their underlying code-level root causes. Our findings reveal that 68% of generated webpages exhibit at least one compatibility issue, underscoring the pervasive reliability concerns surrounding MLLM-generated front-end artifacts. The most prevalent symptoms are failures that disrupt the entire page layout (88.3%): pages shrink directly to fit the target screen with too small fonts, or exhibit scale mismatches that produce cut-off content. Failures localized to individual elements, such as image distortion or missing components, are comparatively less common (13.4%). Furthermore, although most MLLMs incorporate responsive design patterns into the generation, they fail to properly implement these codes.

Guided by the findings, we develop XCOMPAT, a lightweight offline compatibility issue detector that combines visual screenshots and the structural DOM tree for analysis. It achieves an F1 score of 0.903 on the WebCompat-test, outperforming the existing compatibility checking tools and LLM baselines. All datasets and tools are released to support future research on rendering reliability in MLLM-based front-end code generation.

Index Terms—Multimodal code generation, UI2Code, evaluation, cross-environment compatibility

I. INTRODUCTION

Multimodal large language models (MLLMs) have been widely adopted to produce front-end code from screenshots or natural-language requirements [1–4]. This capability has driven the development of numerous AI-assisted web development products, representing a rapidly expanding market [5, 6]. Concurrently, academic research has formalized this problem as the UI-to-code (i.e., screenshot-to-code) task, yielding a growing body of benchmarking [7–9] and methodology studies [10]. Despite competitive benchmark scores, it remains

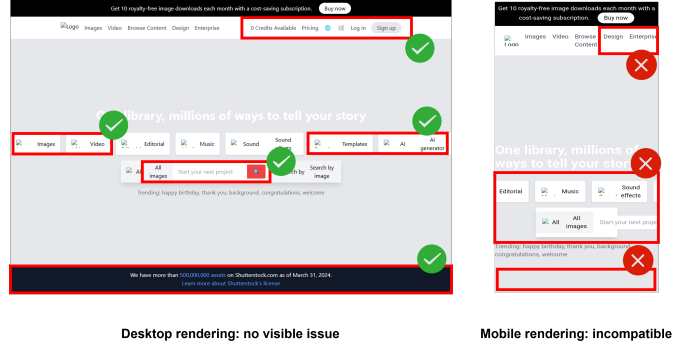


Fig. 1. An example showing that correct rendering in one environment does not guarantee cross-environment compatibility. The generated webpage appears correct in the desktop environment, but exhibits element overflow and visibility issue when rendered in a mobile environment.

unknown whether MLLM-generated front-end code is ready for real-world deployment, where it must render correctly across diverse browsers and devices [11–14].

Existing UI-to-code work [7, 8, 10, 15] emphasizes the generation quality through visual fidelity. Given a visual design (screenshot) as input, UI-to-code asks models to generate the front-end code that could reproduce the design. Current benchmarks then compare an AI-generated webpage with its reference under a *fixed rendering setup*, using metrics such as image similarity, text matching, or code-level similarity [7, 9, 10]. However, real-world webpages are accessed across different *environments*, spanning various devices and browsers. As a result, a page that renders correctly under one configuration may still fail in other environments, manifesting as layout shifts, content overflow, or visual distortion. Figure 1 illustrates this gap: the generated page renders correctly in desktop rendering, but exposes an element overflow and visibility issue in a mobile environment. This raises a natural question: **do AI-generated webpages suffer from cross-environment rendering compatibility issues?**

To answer the question, we conducted the first empirical study on compatibility issues in UI-to-code generation. We defined an incompatibility as a *user-perceptible visual failure* that is not attributable to intended responsive adaptation and

that degrades the webpage’s presentation, content accessibility, or usability. Specifically, we propose WEBCOMPAT, a dataset covering webpages from representative AI tools across three categories: commercial UI-to-code products, frontier MLLMs, and academic open-source systems. We rendered each webpage across 9 environments spanning multiple browsers and devices, yielding 2,032 instances for compatibility analysis. We then examined the prevalence of compatibility issues in these webpages, characterize their visual symptoms, and uncover the underlying causes at the code level.

Our study finds that: **(1)** Incompatibility is pervasive across AI-generated webpages. Across the eight generators, 68% of generated webpages contain at least one cross-environment compatibility issue on average. **(2)** AI-generated webpages fail more often than human-written webpages (40%) under the same rendering settings. **(3)** Cross-device incompatibility dominates cross-browser incompatibility. Among generated webpages, 67.5% exhibit cross-device-only failures, while only 1.0% exhibit cross-browser-only failures. **(4)** The observed failures are mostly page-level. Page-level symptoms account for 88.3% of cases and are dominated by direct page shrink, initial scale mismatch, and bottom whitespace with viewport adaptation issues. **(5)** While the AI-generated pages often include responsive layout design, they still fail due to incomplete responsive implementations. The dominant causes are missing viewport meta (46%) and missing flexible wrapping (29.5%)

Based on empirical findings, we observe that existing compatibility testing tools cover only part of the incompatibilities in MLLM-generated webpages. To automate the comprehensive detection, we design XCOMPAT, a lightweight detector that analyzes screenshots and post-render DOM trees to identify the aforementioned incompatibilities. On the test set, XCOMPAT achieves an F1 score of 0.903 and an accuracy of 0.953 over 408 instances, outperforming all baselines while maintaining high efficiency.

In summary, this paper makes the following contributions.

- **Problem formulation.** We introduce cross-environment compatibility as a quality dimension for AI-generated webpages and formulate it as a rendered output comparison problem.
- **Benchmark.** We construct WEBCOMPAT, a dataset with 2,032 annotated instances of AI-generated webpages from multiple generators and rendering environments.
- **Empirical findings.** We show that compatibility issues are pervasive in AI-generated webpages and further derive five findings on their failure symptoms and causes.
- **Detection method.** We develop XCOMPAT, a lightweight offline detector that outperforms existing compatibility testing tools and multimodal LLM baselines.

II. RELATED WORK

A. Compatibility Testing for Web Applications

Existing research on web compatibility testing has focused on three directions: cross-browser incompatibility (XBI),

cross-device environments (XDI), and responsive layout failures. The earliest work [16] characterized XBI as a pervasive, costly, and understudied defect in web engineering. A line of following work utilizes differential testing to compare DOM structures, visual renderings, and behavioral states across different rendering engines, as demonstrated by tools like WebDiff [17], X-PERT [18], and CrossCheck [19]. Complementarily, Browserbite [20] approaches XBI purely through screenshot comparison, using image features and machine learning to predict incompatibilities without requiring DOM access. The differential paradigm extends naturally to XDI, where device-cloud platforms such as BrowserStack execute visual regressions across a matrix of browser, hardware, and OS configurations. Since it involves more drastic viewport changes, most practices still rely on manual inspection after simulated rendering. More recently, ReDeCheck [21] employs an oracle-free, graph-based analysis to enumerate viewport widths and detect failures in element alignments, flagging cases where the front-end code breaks down.

While prior web compatibility research exclusively evaluates human-authored code, the unique cross-environment failures of AI-generated webpages remain entirely unexplored. Furthermore, existing detection tools target narrow, individual defect classes (such as isolated browser or viewport issues). Our work establishes the first unified benchmark and holistic detection framework for compatibility failures produced by AI coding.

B. UI-to-Code Methods and Evaluations

Translating visual designs into code is challenging and time-consuming, as it requires domain knowledge to map GUI components to spatial layouts. MLLMs have demonstrated strong capability in this task. Benchmarking studies such as Design2Code [7] evaluate models on generating HTML/CSS implementations from webpage screenshots, while generation methods such as DCGen [10], UICopilot [22], and WAF-FLE [23] improve UI-to-code generation through decomposition, hierarchical generation, or structure-aware fine-tuning, respectively. Subsequent work has further advanced MLLMs’ UI-to-code ability through techniques such as layout guidance [1], token compression [24], component-based implementation [25] and test-driven development [26].

However, existing evaluations focus exclusively on reproduction similarity within a fixed rendering environment. Their metrics, such as visual similarity (SSIM), and code similarity (BLEU), measure reproduction fidelity but fail to reflect the cross-environment reliability of generated webpages. Neglecting this dimension poses practical risks when deploying AI-generated pages to real-world scenarios.

III. DATASET CONSTRUCTION

We construct WEBCOMPAT, a dataset for analyzing cross-environment compatibility issues of AI-generated front-end code. It consists of webpages produced by eight representative generation tools, rendered on nine devices and browser

environments through BrowserStack [27], with ground-truth compatibility labels from human annotation.

A. Problem Formulation

We formulate the compatibility issues detection as a pairwise comparison task. For an AI-generated page p , we render it in two environments, namely reference e_r and test e_t . Each rendering yields a screenshot and its post-render DOM, written $R(p, e) = \langle I(p, e), D(p, e) \rangle$. Given the two renderings, the task f is to decide whether the target has a visual difference relative to the reference:

$$f(R(p, e_r), R(p, e_t)) \rightarrow \text{bool}(\text{isCompatible}) \quad (1)$$

B. Construction Pipeline

We aim to gather a comprehensive dataset capturing the landscape of front-end design automation tools and reveal their performance in multi-environment rendering scenarios. To this end, we constructed a dataset through a four-stage pipeline, as illustrated in Figure 2: (1) source data collection, (2) webpage generation, (3) multi-environment rendering, and (4) manual annotation on incompatibility. Each stage is described in detail below.

1) *Stage 1: Source Data Collection*: Our study focuses on single-page instances from established UI-to-code benchmarks, where each page is self-contained in a single file and rendered across web browsers on different devices. We choose single-page static applications because they have stable and deterministic rendering, regardless of user interactions or backend state. This isolates cross-environment rendering issues and allows us to measure compatibility independently.

In particular, we source webpages from two representative datasets, DCGen [10] and Design2Code-Hard [7], both of which provide paired visual inputs and the associated HTML/CSS implementations. The two datasets differ in both design intention and task difficulty: DCGen is built from real-world websites, while Design2Code-Hard is a harder dataset that deliberately incorporates difficult cases from GitHub Pages, including long pages, intricate HTML structures, and non-English content, which require models to handle more difficult visual-text recognition and layout reproduction. We randomly sample 30 pages from each, resulting in 60 source pages for the subsequent generation stage.

2) *Stage 2: Webpage Generation*: To reflect current practice in automated front-end development, we employ a diverse set of leading UI-to-code methods to reproduce the source pages, spanning frontier LLMs, commercial agentic coding tools, and academic representatives. Concretely, we include three frontier LLMs (GPT-4 [28], GPT-5.1 [29], and Gemini-3-Flash [30]), two agentic coding tools (v0.dev [5] and Cursor [6]), and three variants of DCGen that use different LLM backends (DCGen-GPT4, DCGen-GPT5.1, and DCGen-Gemini-3-Flash). DCGen is a leading academic tool based on the current leaderboard [31]. To ensure a fair comparison, we standardize both prompt design and tool configuration. For DCGen, we follow its original implementation. For all other

TABLE I
DEVICE MATRIX (BROWSERSTACK)

Device	OS	OS Ver.	Browser	Ver.
<i>Mobile</i>				
Google Pixel 7	Android	14	Chrome	146.0
iPhone 13	iOS	18	Safari	15.6
iPhone 13	iOS	18	Chrome	146.0
<i>Desktop — Viewport 1280×800, Screen 1920×1080</i>				
Desktop	Windows	11	Edge	145.0
Desktop	Windows	11	Chrome	146.0
Desktop	Windows	11	Firefox	148.0
Desktop	OS X	Monterey	Chrome	146.0
Desktop	OS X	Monterey	Firefox	148.0
Desktop	OS X	Monterey	Safari	15.6

prompt-based generators, we use the same prompt template per instance (adopted from DCGen’s direct-prompting variant). For agentic coding tools, we keep their default configurations to reflect their standard usage conditions. This process yields $60 \times 8 = 480$ candidate webpages for the next stage.

3) *Stage 3: Multi-Environment Rendering*: This stage renders each generated page in multiple environments to expose its compatibility issues. To emulate a variety of real-world webpage access scenarios, we consider two dimensions: cross-browser and cross-device. First, different browsers (e.g., Google Chrome, Firefox, Safari) use different layout calculations, spacing rules, and overflow handling in the rendering engine, all of which may lead to visual inconsistencies. Second, different devices have different viewport sizes, where a **viewport** refers to the visible area of a webpage on a screen. A webpage that displays correctly on a desktop browser may fail on a mobile device due to its smaller viewport. This can lead to layout issues like element overlap or clipping. Therefore, we include cross-device environments to capture failures that depend on viewport differences.

Following this design, we render each webpage across nine device-browser environments on BrowserStack [27], a cloud platform that runs real devices and browsers. Table I lists the corresponding operating systems and browser configurations.

After rendering, we remove pages that are not suitable for cross-environment comparison. These include pages with blank content, malformed HTML output, failures during rendering and capturing. We also remove layouts that depend on unsupported interactive behavior such as collapsible menus. This filtering retains 254 valid webpages. Then, we set Chrome-on-Windows as the reference environment, because Chrome’s Blink engine represents the most widely deployed browser, and desktop applications remain the dominant platform in existing GUI research [17, 21, 32, 33]. For each reference, we pair it with each of the other eight renderings as target environments. This produces $254 \times 8 = 2,032$ cross-environment comparison pairs for annotation.

4) *Stage 4: Incompatibility Issue Annotation*: Annotation is conducted by two annotators with at least four years of experience in front-end web development and GUI testing.

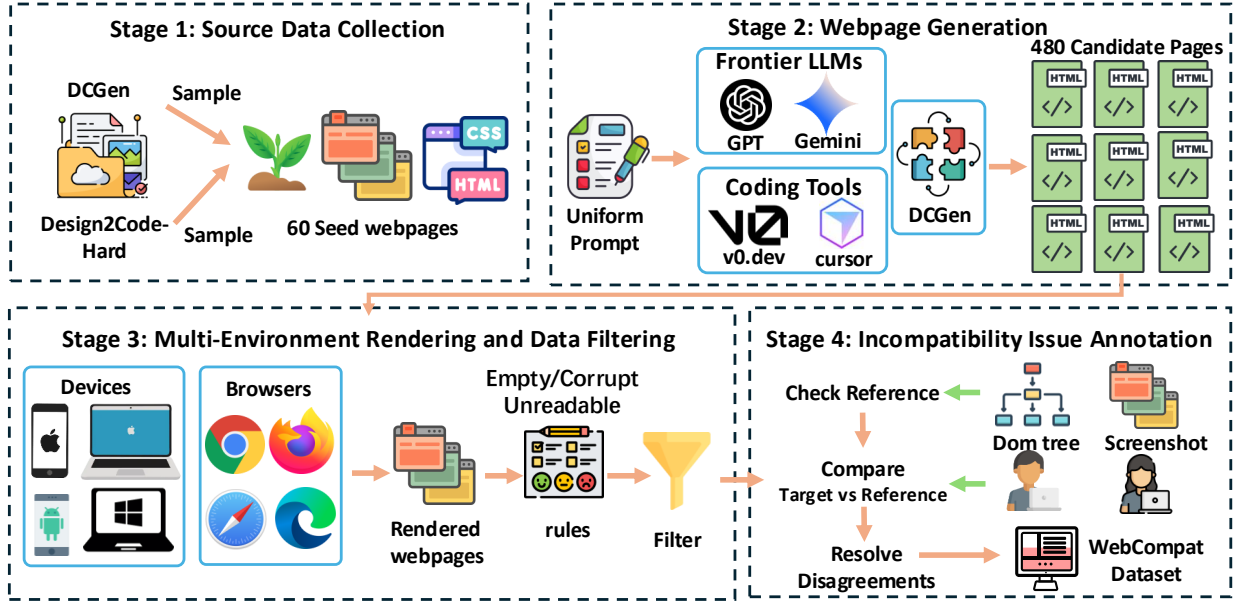


Fig. 2. Overview of the WebCompat construction pipeline.

Algorithm 1 Cross-environment compatibility annotation

Require: Renderings $R(p, e_r)$ and $R(p, e_t)$

Ensure: COMPATIBLE or INCOMPATIBLE

```

1: if  $e_r$  and  $e_t$  share the same device then
2:   compare elements across  $D(p, e_r)$  and  $D(p, e_t)$  by position, size,
   and appearance
3:   if any element differs then return INCOMPATIBLE
4: end if
5: else
6:   if the layout in  $R(p, e_t)$  reflows relative to  $R(p, e_r)$  then
7:     if any element is missing, overflows the viewport, or overlaps
       another then return INCOMPATIBLE
8:   end if
9:   else
10:    if the scaled layout is distorted then return INCOMPATIBLE
11:    end if
12:  end if
13: end if
14: return COMPATIBLE

```

Prior to the formal annotation phase, they complete a pilot labeling session on a sampled set to align their understanding of compatibility issues. In the main phase, each rendering pair was first labeled independently by the two annotators following the procedure in Algorithm 1. The independent annotations achieve a Cohen’s kappa of 0.9506, indicating a high level of inter-annotator agreement. Then, disagreements between the two annotators are resolved through a third adjudication round conducted through consensus discussion. All disagreements are resolved at the end of the discussion.

C. Dataset Summary

Our dataset comprises 254 valid webpage generations and 2,032 annotated instances (i.e., rendering pairs) across multiple environments. Figure 3 shows the distribution of instances across the AI coding tools. Together, the diversity of websites,

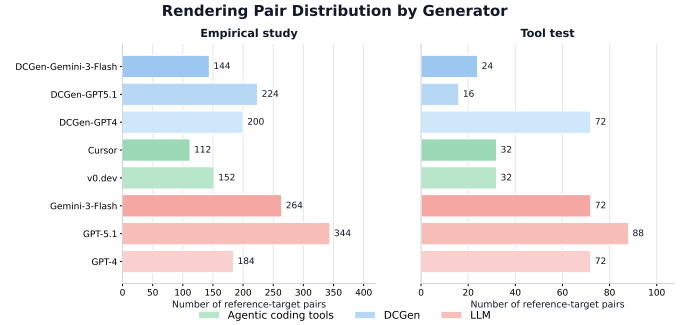


Fig. 3. Distribution of rendering pairs across the various generators.

AI coding tools, and rendering environments provides the foundation for our study of compatibility issues.

We partition the data 8:2 (i.e., 203:51) at the page level. The larger split is used for empirical analysis, while the held-out 20% serves as a test set for evaluating automated compatibility issue detection to avoid data leakage.

IV. EMPIRICAL CHARACTERIZATION

Building on the dataset described in Section III, we structure our empirical study around the following research questions:

- **(RQ1)** *How prevalent are compatibility issues in AI-generated webpages?* We measure generation fidelity and compatibility rates, and compare incompatibility prevalence against human-authored counterparts.
- **(RQ2)** *What symptoms do these compatibility issues manifest?* We manually analyze compatibility issues and derive a taxonomy of cross-environment failure symptoms.
- **(RQ3)** *What are the root causes of these compatibility issues?* We perform code-level analysis to identify sources

of rendering inconsistency, yielding insights into subpar coding patterns through MLLM generation.

A. RQ1: Prevalence of Compatibility Issues

1) The Compatibility issues in AI-generated webpages.

Motivation. The existing UI2Code evaluation compares the reproduction similarity with the original design in a single environment. However, a competitive score under such metrics does not reflect whether the same page remains reliable across different browsers and devices. Therefore, we first examine the prevalence of compatibility issues in these AI reproductions.

Settings. We study the relationship between UI fidelity scores and incompatibility rates on 203 AI-generated webpages from the empirical-study split. Following previous UI-to-Code work [7, 31], we use multiple UI fidelity metrics to cover different aspects of webpage reproduction quality. We report visual similarity metrics such as CLIP [34] and SSIM [35] for appearance, BLEU [36] for textual similarity, code similarity [31] for HTML/CSS. We also report fine-grained visual matching metrics [7] for element-level quality, and MLLM-as-a-Judge based on WebDevJudge [37] for overall UI quality assessment.

Results. Table II shows that incompatibility is pervasive across all generations, despite their high visual fidelity score. On average, 68% of AI-generated webpages contain at least one cross-environment compatibility issue. Across all eight tools, the incompatibility range is from 26% to 100%. The best-performing v0 stands at 0.26, likely owing to its engineering focus on deployment-ready code. This demonstrates that existing AI-produced webpages cannot reliably guarantee cross-environment compatibility.

Across all generators, stronger performance on visual fidelity metrics does not indicate lower incompatibility rates. For example, DCGen-GPT4 achieve the two highest fine-grained visual scores among all models, yet carry a higher incompatibility rate of 0.84. The weak relationship between visual fidelity scores and cross-environment robustness indicates that current benchmarks are insufficient indicators for deployment quality. Although academic models achieve competitive visual fidelity, this does not extend to cross-environment robustness, whereas the commercial tool v0 strikes the most favorable compatibility-fidelity trade-off. This reveals that cross-environment compatibility is a critical quality gap that AI code generation tools must address for real-world deployment.

Finding 1: Incompatibility is pervasive in AI-generated webpages (average 68%). Existing visual fidelity scores show weak correlation with cross-environment robustness.

2) The comparison between AI-generated webpages and human-authored ones.

Motivation. To understand how widespread compatibility issues are in practice, we compare MLLM generation results with human-written ones, and analyze the primary sources of compatibility challenges.

Settings. Using the 203 valid AI-generated webpages from the empirical-study split, we compare them with 60 human-authored implementations from Design2Code and Design2Code-HARD [7], which were created before the recent wave of LLM-based webpage generation. We exclude DCGen source pages from the human baseline because many rely on JavaScript-driven behavior, which is outside our static-page setting. We render the human-authored pages using the same multi-environment pipeline and pairwise annotation procedure in §III-B3 and §III-B4, respectively. The two annotators achieve a Cohen’s kappa of 0.943 on the human pair-level labels.

Results. AI-generated webpages fail about $1.7\times$ as often as human-written webpages. As shown in Table III, 139 of 203 AI-generated webpages contain at least one compatibility issue, yielding a failure rate of 68%. The human-written baseline has 24 incompatible webpages out of 60, yielding a failure rate of 40%. A Fisher exact test confirms that this difference is statistically significant, $p < 0.001$. This result suggests that cross-environment compatibility is already a common and difficult problem in human web development, while AI-generated webpages are even more likely to suffer from such issues. This gap also appears across the two source groups in the AI set. Design2Code-HARD-based generations fail on 74 of 95 webpages, while DCGen-based generations fail on 65 of 108 webpages.

Finding 2: AI-generated webpages exhibit $1.7\times$ more cross-environment issues than human-written webpages.

We further go in-depth into cross-browser incompatibility (XBI) and cross-device incompatibility (XDI), because they stress different aspects of front-end compatibility. XBI captures browser-dependent rendering differences under desktop viewport settings. XDI captures whether the same page adapts from desktop to mobile viewport settings. Figure 4 shows their distribution. XDI (blue bar) is substantially more prevalent than XBI across both groups. Among AI-generated webpages, 67.5% exhibit XDI-only failures, while only 1.0% exhibit XBI-only failures. Among human-authored webpages, 36.7% show XDI-only failures, compared to just 1.7% with XBI-only failures and 1.7% with both. The remaining 36 human-authored pages (60.0%) are issue-free. These XDI-dominant results indicate the viewport size difference is a greater rendering challenge. While modern browsers have standardized rendering engines, adapting layouts across drastically different screen sizes (e.g., 320px to 1920px) demands comprehensive responsive design logic.

Finding 3: Cross-device incompatibility dominates cross-browser incompatibility in both AI-generated and human-written webpages.

TABLE II

GENERATION QUALITY OF EACH GENERATOR ON THE EMPIRICAL STUDY SET. BEST SCORE IN EACH ROW IS **BOLD**. *Overall UI Quality* HAS A MAXIMUM SCORE OF 15. THE AVERAGE COLUMN REPORTS A SAMPLE-WEIGHTED MEAN, WHERE THE **WEIGHTS ARE THE NUMBERS OF RENDERING PAIRS** PRODUCED BY EACH GENERATOR.

Metric	Agent		DCGen			Direct			Average
	Cursor	v0	DCGen-Gemini3	DCGen-GPT4	DCGen-GPT5	Direct-Gemini3	Direct-GPT4	Direct-GPT5	
Visual / Structural Similarity									
CLIP	0.8955	0.8715	0.9030	0.8441	0.8997	0.8738	0.8204	0.8947	0.8760
SSIM	0.7807	0.7889	0.7780	0.7899	0.8138	0.7994	0.7807	0.7870	0.7913
BLEU	0.6157	0.7360	0.7112	0.5463	0.7290	0.7178	0.3811	0.6941	0.6491
Structure Code Similarity	0.0597	0.1370	0.0812	0.1012	0.0814	0.0916	0.1162	0.0796	0.0928
Fine-grained Visual									
BlockMatch	0.3317	0.7553	0.7676	0.6821	0.6249	0.5606	0.6666	0.7053	0.6479
Text	0.6736	0.9068	0.8752	0.9719	0.8278	0.8958	0.9581	0.9406	0.8962
Position	0.5792	0.8155	0.7821	0.8477	0.7568	0.7906	0.7811	0.8386	0.7890
Color	0.6161	0.8352	0.8117	0.8514	0.7817	0.7918	0.8530	0.8742	0.8159
MLLM-as-a-Judge									
Overall UI Quality	7.286	6.895	9.167	6.360	7.714	9.424	6.174	8.558	7.852
Incompatibility Rate	0.79	0.26	0.50	0.84	0.93	0.42	0.43	1.00	0.68

TABLE III

CROSS-ENVIRONMENT INCOMPATIBILITY RATES OF AI-GENERATED WEBPAGES AND THE HUMAN-WRITTEN BASELINE. INCOMPATIBILITY RATES ARE COMPUTED PER WEBPAGE, NOT PER RENDERING PAIR.

Category	AI Tools	Design2Code	DCGen	Combined
Agent	Cursor	5/6 (83%)	6/8 (75%)	11/14 (79%)
	v0	3/7 (43%)	2/12 (17%)	5/19 (26%)
DCGen	Gemini-3-Flash	4/8 (50%)	5/10 (50%)	9/18 (50%)
	GPT-4	11/12 (92%)	10/13 (77%)	21/25 (84%)
	GPT-5	14/14 (100%)	12/14 (86%)	26/28 (93%)
Direct	Gemini-3-Flash	8/15 (53%)	6/18 (33%)	14/33 (42%)
	GPT-4	7/11 (64%)	3/12 (25%)	10/23 (43%)
	GPT-5	22/22 (100%)	21/21 (100%)	43/43 (100%)
AI overall		74/95 (78%)	65/108 (60%)	139/203 (68%)
<i>Human-written baseline</i>		24/60 (40%)		

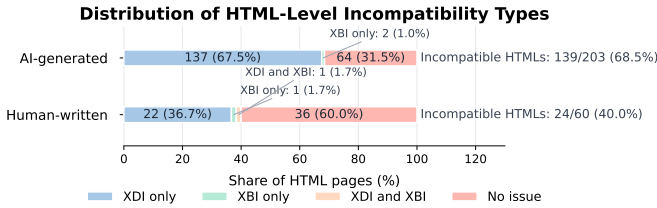


Fig. 4. Distribution of XBI, XDI, and issue-free pages for human-written and AI-generated webpages.

B. RQ2: Failure Symptoms

Motivation. While RQ1 shows that most AI-generated webpages have compatibility issues, their binary label does not tell *how* it fails. We therefore study the visual and layout symptoms in incompatible pages to understand their user-perceived impact.

Settings. Two annotators first inspected screenshots from 20% of the incompatible webpages as a pilot set. Before the pilot annotation, we provided the annotators with an initial taxonomy adapted from prior work (ReDeCheck [21] and WebDiff [17]). During the pilot annotation, they refined this taxonomy into two failure levels (page-level, element-level) and derived an initial set of issue classes. A page-level failure affects the overall layout structure, whereas an element-level failure remains localized to an individual element. Upon this, the two annotators then independently labeled the full set of incompatible cases. When they encountered a new issue class, they discussed it, added it to the guide, and applied it consistently to the remaining cases. Disagreements were collected and resolved through discussion. Before this resolution, the inter-annotator agreement was 0.825, measured by macro-averaged Cohen’s κ [38] over all incompatible pairs.

Results. Table IV presents the failure symptom taxonomy, spanning two class levels (page-level, element-level) with eight failure types, their definitions, and distribution. One instance may exhibit multiple failure symptoms. Compatibility failures in AI-generated webpages are predominantly page-level: 88.3% of incompatible pairs exhibit page-level failures, compared to 13.4% with element-level failures (note: instances may exhibit multiple failure symptoms).

Page-level failures are further divided into two subtypes: *viewport adaptation* (A1) and *structural layout failure* (A2). Viewport adaptation failure occurs when the page preserves desktop element spatial relationships despite being rendered on smaller viewports, causing cramped layouts and unreadable content (Fig. 5). In contrast, structural layout failure covers cases where the page *attempts* to restructure for mobile but produces broken layouts (Fig. 6). Element-level failures (Class B) are localized to individual components: image size dis-

TABLE IV
TAXONOMY OF CROSS-ENVIRONMENT FAILURE SYMPTOMS IN AI-GENERATED WEBPAGES. CLASS A FAILURES DISRUPT GLOBAL PAGE LAYOUT;
CLASS B FAILURES ARE LOCALIZED TO INDIVIDUAL ELEMENTS.

Level	Type	Definition	%
Class A: Page-level failures			88.3%
A1 (Viewport adaptation failure)	(A1.1) Shrink-to-Fit	Page directly shrinks to fit the target screen, making the content too small to read.	42.1
	(A1.2) Initial Scale Mismatch	Initial page scale does not match the device width, causing content to be cut off horizontally or leaving the viewport width only partially occupied.	23.1
	(A1.3) Whitespace Anomaly	Unexpected large blank gap at the page bottom after rendering (absent from reference).	13.8
A2 (Structural layout failure)	(A2.1) Overflow	Content extends beyond the container/viewport or gets clipped.	23.1
	(A2.2) Overlap	Elements collide and stack with each other, hiding text or controls.	5.5
Class B: Element-level failures			13.4%
B	B1 Image Size Distortion	Image/placeholder is stretched relative to its intended dimensions.	10.8
	B2 Visibility Failure	Missing elements in target rendering.	2.1
	B3 Appearance Deviation	Element correctly positioned but missing or altered CSS styles change its visual appearance such as color or styling.	0.5

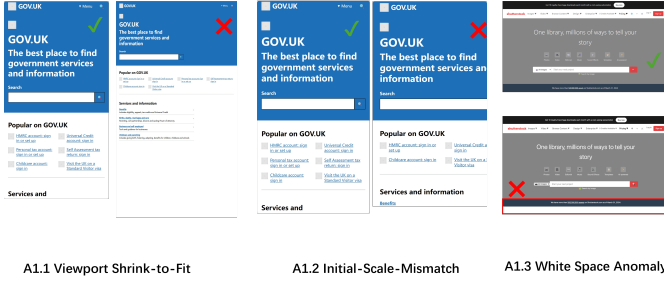


Fig. 5. Subclass of A1 (Viewport adaptation failure).

tortion (B1), visibility failure (B2), and appearance deviation (B3), as illustrated in Fig. 7. In many cases, the generated webpages behave like desktop-oriented layouts and do not reflow according to the smaller viewport size of mobile devices.

Finding 4: Compatibility failure in AI-generated webpages are mostly page-level (88.3%) with viewport adaptation issues.

C. RQ3: Root Causes of Compatibility Issue

Motivation. Visible compatibility failures show how a page breaks. They do not explain *why*, namely, which code patterns cause the breakage. The same viewport adaptation overflow failure may have different causes. Examples include a missing viewport declaration, fixed dimensions, and incomplete wrapping rules. We therefore examine the recurring HTML/CSS patterns behind these failures.

Settings. We analyze the 139 incompatible AI-generated webpages identified in RQ1. For each page, the same two annotators (§III-B4) inspect its HTML, CSS, and rendered screenshots, and independently assign one or more root-cause labels. We allow multiple labels because a page may contain several code defects, such as a missing viewport meta tag

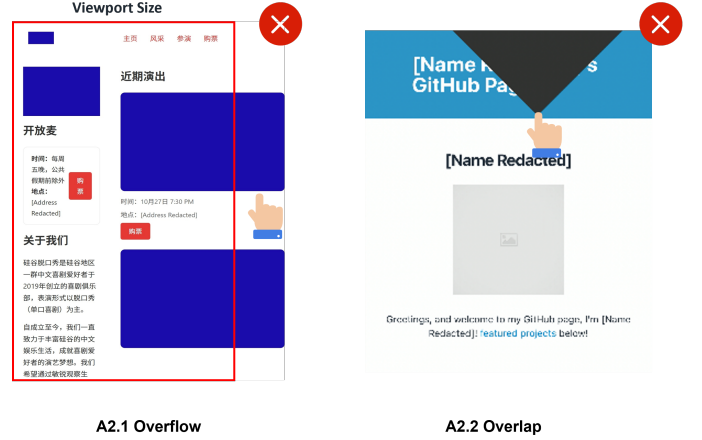


Fig. 6. Subclass of A2 (Structural Layout Failure): A2.1 shows content overflow when elements exceed the viewport; A2.2 shows overlap caused by elements colliding after responsive reflow.

together with hard-coded widths. We measure agreement with per-category binary Cohen’s kappa and obtain a macro-average kappa of 0.940 before resolving disagreements through discussion. We derive the categories from observed code patterns.

In addition, we conduct a keyword scan to examine the usage of responsive layout constructs. Responsive constructs are CSS/HTML mechanisms that enable a webpage to adapt its layout to different screen sizes and rendering environments. These include `flex`, `grid`, and Tailwind prefixes such as `sm:`, `md:`, and `lg:`. The scan provides context for interpreting whether failures occur despite the presence of responsive layout constructs.

Results. Responsive constructs are widely-used in the generated code, but many pages still fail across rendering environments. In the current keyword scan, 97.5% of instances contain at least one responsive layout construct. The keyword `flex` appears in 95.1% of the instances, and the keyword `grid` appears in 43.3%. At the same time, 139 generated pages in the

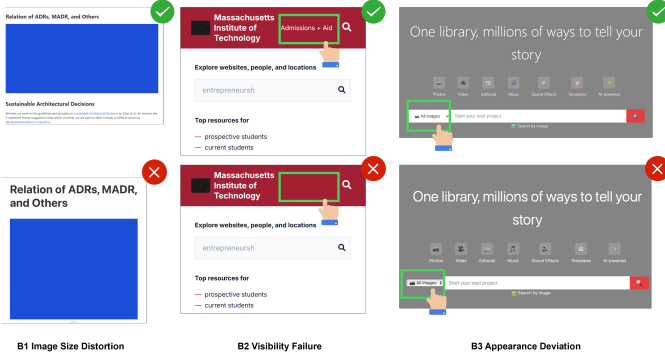


Fig. 7. Examples of Class B element-level incompatibilities: B1 (image size distortion), B2 (visibility failure), and B3 (appearance deviation) in AI-generated webpages.

TABLE V
CODE-LEVEL ROOT CAUSES AMONG INCOMPATIBLE AI-GENERATED WEBPAGES.

Root cause	#Pages	Rate
Missing viewport meta	64	46.0%
Missing flexible wrapping	41	29.5%
Hard-coded dimensions	27	19.4%
Hard-coded image size	14	10.1%
Hidden element	3	2.2%
Browser rendering anomaly	2	1.4%

empirical study set remain incompatible across environments. This contrast indicates that MLLMs are aware of responsive constructs in front-end coding, but *their implementation of these constructs is incomplete or incorrect*. Table V reports six code-level root causes that turn responsive code into compatible failures. We detail them below.

Missing viewport meta (46.0%). Pages that omit or misuse `<meta name="viewport">` fail to specify intended layout width to mobile. So mobile browsers default to rendering the page at desktop viewport width, then scale it down to fit the physical screen. This shrinking causes text or elements to be too small, manifesting as symptoms A1.1 or A1.2.

Missing flexible wrapping (29.5%). Pages implement `flex` or `grid` but omit `flex-wrap` or equivalent wrapping rules. In such a case, containers force child elements to remain on a single line, even when the viewport is too narrow, mostly causing A2.

Hard-coded dimensions (19.4%). Fixed layout values lock element sizes (`min-width`, `width: ...px`). On smaller viewports, such values either exceed available space (causing overflow) or fail to scale responsively, leading to A1.2 or A2.1.

Hard-coded image size (10.1%). Responsive width utilities `w-full` adapt to viewport width, but pairing with fixed heights or aspect-ratio assumptions that don't scale. The image stretches disproportionately, causing B1.

Hidden element (2.2%). Responsive visibility utilities (`hidden lg:block`) hide elements on mobile without offering a replacement entry point. As a result, the hidden

elements become unreachable in the mobile rendering, leading to B2.

Browser rendering anomaly (1.4%). Browser-specific defaults in styling, spacing, or native control rendering diverge across browsers and mainly lead to B3.

Finding 5: While AI-generated webpages often include responsive layout constructs, they fall short of correct implementation. The dominant causes are missing viewport meta (46%) and missing flexible wrapping (29.5%).

V. XCOMPAT: COMPATIBILITY DETECTION TOOL

Our empirical study shows that compatibility failures in AI-generated webpages are common. To detect these failures before production, we propose an automated, lightweight compatibility issue detection tool, XCOMPAT, based on post-rendered visual signals. We formulate the detection task at the same pair granularity as in previous sections.

A. XCompat Design

For each reference-target pair, XCOMPAT first extracts signals from the rendered DOM and screenshot, applies the two analysis modules, and then combines their module results into a single pair-level compatibility decision. Specifically, it uses a Page-Level Analysis Module for global layout and viewport problems and an Element-Level Analysis Module for localized component problems.

To start with, XCOMPAT utilizes post-render HTML DOM tree, screenshots, and viewport metadata. This information will be fed into the two modules below.

Page-Level Analysis Module. This module mainly detects (Class A) page-level failures, including viewport adaptation and structural layout failures. Motivated by the finding that a large proportion of incompatibilities stem from missing or misconfigured viewport metadata, we start by inspecting code-level viewport configurations, including default width settings, target viewport-width, content fill ratio, and horizontal overflow, using heuristic predicate thresholds. For structural analysis, it identifies whitespace anomalies by detecting abnormal blank areas in the target screenshot but absent in the reference. It further detects layout collapse by computing overlaps among multiple elements in the rendered DOM tree. A substantial discrepancy in overlaps between rendering environments indicates conflicting global layout structures and is flagged as a compatibility failure.

Element-Level Analysis Module. This module targets localized compatibility failures on individual matched DOM elements across rendering environments (Class B). Taking per-element rendering signals such as bounding boxes, aspect ratios, and visibility state as inputs, the module detects two classes of failures motivated by our incompatibility taxonomy. First, it identifies distorted elements by comparing aspect ratios between matched elements; a substantial deviation indicates abnormal stretching or compression caused by environment-specific rendering. Second, it captures visibility inconsistencies by examining rendering-specific states such as `display`

or opacity; discrepancies in these signals reveal elements that are unintentionally changed between renderings.

Aggregation. The two above modules generate results at different granularities. XCOMPAT finally combines them with a logical operator OR. A pair is marked INCOMPATIBLE if either module returns a compatibility failure.

B. Experiment Setup

We evaluate XCOMPAT on the following research questions:

- **(RQ4)** *How accurately does XCOMPAT detect cross-environment incompatibilities?* We compare it with existing compatibility checking tool and LLM-based baselines.
- **(RQ5)** *How efficient is XCOMPAT?* We conduct the runtime and monetary cost for our tool.

1) *Dataset:* Section III is split 8:2 into an empirical set and a test set. The empirical set supports design intuitions, threshold selection, prompt selection for LLM baselines, and baseline sanity checks. To avoid data leakage, we fix all configurations and separately evaluate XCOMPAT on the 20% unseen test set. We also report the performance difference between the two sets.

2) *Baselines:* We compare XCOMPAT with existing compatibility testing tools and two LLM-based baselines. We do not include X-PERT or WebDiff as primary baselines because they primarily target cross-browser inconsistencies at a fixed viewport, whereas our benchmark is dominated by cross-device responsive-layout issues, as shown in RQ1. REDECHECK [21] analyzes layout structure during viewport resizing and serves as a DOM-geometry-based compatibility baseline. For LLM-based baselines, we evaluate two input settings for each model to assess whether visual rendering evidence improves compatibility issue detection. For both settings, we provide the same taxonomy definitions used in our annotation protocol. The *DOM-only setting* receives post-render DOM snapshots, environment names, and the comparison mode. The *DOM w/ screenshot setting* additionally receives the reference and target screenshots. We choose GPT-5.5 and Claude Opus 4.8 for comparison.

3) *Metrics:* Following prior work on cross-browser incompatibility and responsive-layout failure detection [18, 21, 39], we report *Precision*, *Recall*, and *F1* for incompatibility detection results. We additionally report *Accuracy* since our detection task is evaluated as pair-level binary classification. For efficiency, we report *execution time per comparison* and estimated *monetary cost* per 1,000 comparisons when using online model providers.

C. RQ4: Effectiveness of XCompat

Table VI reports detection results on both the empirical set and the test set. The empirical set is used for empirical analysis in Section IV, while the test set is entirely unseen, demonstrating XCOMPAT’s effectiveness on unseen webpages.

XCOMPAT achieves the highest F1 on both sets. On the test set, XCOMPAT achieves Precision of 0.917, Recall of 0.889, F1 of 0.903, and Accuracy of 0.953 over 408 instances, meaning it can detect 91% compatibility issues in AI-generated

webpages. On the empirical set, XCOMPAT reaches F1 of 0.939, compared with 0.892 for the strongest LLM baseline and 0.565 for REDECHECK. The modest F1 drop of 3.6% from the empirical set to the unseen test set indicates that the fixed detector generalizes well on unseen pages and remains stronger than both structural and LLM-based baselines.

The ablation study demonstrates that both page-level and element-level modules effectively contribute to our method (F1 score). The greater performance drop without the page-level module corroborates our finding of dominant page-level issues. The page-level-only variant prioritizes precision but reduces recall and overall F1 performance due to increased false negatives. We also conduct a sensitivity analysis for three manually specified predicate thresholds on the test set. Varying the viewport-width threshold from 800 px to 760 and 840 px, the content-fill threshold from 0.75 to 0.70 and 0.80, and the mobile wide-content threshold from 2.0 to 1.8 and 2.2 does not change any pair-level binary prediction.

In MLLM baselines, we observe that DOM-only inputs already provide useful evidence for compatibility classification. Adding screenshot inputs reduces performance for both models. GPT-5.5 drops from 0.851 to 0.779 F1 on the test set. Opus 4.8 drops from 0.856 to 0.728 F1 on the test set. A likely explanation is that full-page screenshots introduce high-dimensional visual noise and multiple artifacts (responsive changes, scaling effects, visually salient but harmless differences) that distract from clearer DOM signals on viewport size, overflow flags, and element geometry.

D. RQ5: Efficiency and Cost

RQ5 evaluates whether XCOMPAT is practical for repeated multi-environment compatibility testing by analyzing its execution costs. Table VII reports the execution time and cost on the test set, compared to baselines.

XCOMPAT runs as a deterministic offline analysis over pre-captured renderings, eliminating network dependencies and API token consumption. It processes each sample in 0.127 seconds, approximately 51 times faster than the fastest LLM baseline. LLM baselines further incur 15,304 to 26,721 tokens per API call, corresponding to \$0.0934 to \$0.1407 per comparison. Although ReDeCheck also runs offline and requires 0.068 seconds per comparison, as shown in RQ4, this speed comes at the cost of substantially lower detection effectiveness.

E. Failure Study

We further analyzed XCOMPAT errors on the test set. Among the 408 reference–target pairs, XCOMPAT makes 19 errors, including 11 false negatives and 8 false positives.

The false negatives mostly correspond to subtle compatibility problems that are hard to detect automatically. Five cases involves content extending beyond the screen width, where our detector neglects evidence of overflow due to conservative thresholds. Three cases visually appears to contain a desktop layout without proper mobile adjustment, where main content was narrow and centered, but it does not meet the overflow pattern in our detector. The remaining false negatives include

TABLE VI
INCOMPATIBILITY DETECTION RESULTS ON THE EMPIRICAL AND TEST SETS.

Method	Empirical Set ($N = 1,624$)				Test Set ($N = 408$)			
	P	R	F1	Acc	P	R	F1	Acc
<i>Compatibility testing tools</i>								
REDECHECK	0.411	0.904	0.565	0.651	0.342	0.657	0.450	0.610
<i>LLM-based baselines</i>								
GPT-5.5 DOM-only	0.926	0.828	0.874	0.940	0.899	0.808	0.851	0.931
GPT-5.5 DOM w/ screenshot	0.733	0.899	0.808	0.893	0.693	0.889	0.779	0.877
Opus 4.8 DOM-only	0.926	0.860	0.892	0.948	0.909	0.808	0.856	0.934
Opus 4.8 DOM w/ screenshot	0.828	0.710	0.765	0.890	0.788	0.677	0.728	0.877
XCOMPAT (Ours)	0.932	0.946	0.939	0.969	0.917	0.889	0.903	0.953
<i>Ablation study</i>								
XCOMPAT w/o Page-Level Module	0.793	0.160	0.266	0.779	0.905	0.192	0.317	0.799
XCOMPAT w/o Element-Level Module	0.964	0.848	0.902	0.954	0.929	0.798	0.859	0.936

TABLE VII
RUNTIME AND TOKEN COST ON THE TEST SET. RUNTIME IS REPORTED AS MEAN EXECUTION TIME PER COMPARISON OVER 408 TEST-SET PAIRS. TOKEN COST REPORTS MEAN TOTAL API TOKENS PER LOGGED LLM CALL. OFFLINE TOOLS DO NOT CONSUME API TOKENS.

Method	$\bar{t}$ / Comp.	Total Tok. / Call
REDECHECK	0.068 s	0
GPT-5.5 DOM-only	17.53 s	15,304
GPT-5.5 DOM w/ screenshot	27.96 s	20,721
Opus 4.8 DOM-only	6.49 s	22,050
Opus 4.8 DOM w/ screenshot	8.45 s	26,721
XCOMPAT	0.127 s	0

two image-size distortions and one whitespace anomaly, where the visual difference is too minor to be distinguished by the current DOM and screenshot signals. For the false positives, XCOMPAT detects minor coordinate changes, but the changes are visually negligible under our annotation criteria.

Overall, these errors show that XCOMPAT fails in two main situations. First, it may miss compatibility issues when the visual defects in DOM and screenshot are subtle. Second, it may over-report benign geometric changes that resemble compatibility failures but do not affect the perceived layout.

VI. THREATS TO VALIDITY

Source page selection. Our empirical chooses self-contained and static webpages. This is because they provide deterministic comparison, where rendering behavior is reproducible across environments without depending on user interactions or backend state. Nevertheless, the detector operates on rendered screenshots and post-render DOMs, making it technically applicable to any webpage rendering comparisons with the same UI state. We note that responsive design principles apply across framework-based development (Vue, React). While CSS and UI frameworks provide responsive assistants, the final behavior still depends on how framework classes and custom CSS are used. Therefore, our findings remain relevant to framework-based projects.

Prompt formulation. We used consistent, neutral prompts across all AI tools without explicit instructions for cross-environment compatibility, ensuring fair comparison without special prompt engineering effects. This reflects a baseline performance. Prompts emphasizing cross-environment compatibility could improve results but risk over-specialized layouts.

Annotation Subjectivity. A construct threat is the subjectivity of compatibility labeling. Cross-environment rendering naturally introduces visual differences, and some of them are acceptable responsive adaptations. We mitigate this threat by defining compatibility issues operationally as user-perceivable failures affecting visibility, layout, or readability. We further standardize the annotation process with a detailed guideline and pilot labeling involving two independent annotators. Disagreements are resolved through discussion, and we report Cohen’s κ to quantify inter-annotator agreement.

Missing design intents. Another source of ambiguity is missing design intent. Some visual differences may be deliberate, such as image cropping for a full-bleed background or logo clipping within a constrained container. We treat responsive rearrangement as valid behavior and label size/appearance changes as failures only when they cause abnormal distortion or missing styles. Since borderline cases remain subjective, element-level findings should be interpreted conservatively.

VII. CONCLUSION AND FUTURE WORK

We presented the first systematic study of cross-environment compatibility in AI-generated webpages. Using WEBCOMPAT, we found that compatibility issues are widespread, with cross-device failures occurring far more frequently than pure cross-browser failures. Most issues affect page-level layout, readability, and viewport adaptation, and are associated with recurring implementation patterns such as hard-coded dimensions, missing viewport settings, unflexible wrapping, and rigid image sizing. Based on these findings, we developed XCOMPAT, which combines screenshots and DOM snapshots to detect compatibility issues. It outperforms existing structural and multimodal LLM baselines. Our results demonstrate that

single-environment visual fidelity is insufficient for evaluating AI-generated webpages. Future work will extend the benchmark to more interactive and production-oriented webpages and explore automated root cause localization and repair.

VIII. DATA AVAILABILITY

The artifact is available at: <https://github.com/ZiyunGuo/WebCompat>.

REFERENCES

- [1] F. Wu, C. Gao, S. Li, X.-C. Wen, and Q. Liao, "Mllm-based ui2code automation guided by ui layout information," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSSTA, pp. 1123–1145, 2025.
- [2] Y. Chen, X. Yu, S. Ding, Y. Zhang, C. Shi, J. Du, and L. Chen, "Designcoder: hierarchy-aware and self-correcting ui code generation with large language models," *Information and Software Technology*, p. 108214, 2026.
- [3] S. Yun, H. Lin, R. Thushara, M. Q. Bhat, Y. Wang, Z. Jiang, M. Deng, J. Wang, T. Tao, J. Li *et al.*, "Web2code: A large-scale webpage-to-code dataset and evaluation framework for multimodal llms," *Advances in neural information processing systems*, vol. 37, pp. 112 134–112 157, 2024.
- [4] M. Yuan, J. Chen, Z. Xing, A. Quigley, Y. Luo, T. Luo, G. Mohammadi, Q. Lu, and L. Zhu, "Designrepair: Dual-stream design guideline-aware frontend repair with large language models," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 2483–2494.
- [5] Vercel, "Screenshots and Files," [Online]. Available: <https://v0.app/docs/screenshots>, accessed: Jun. 18, 2026.
- [6] Cursor, "Build Software with AI Agents," [Online]. Available: <https://cursor.com/product>, accessed: Jun. 18, 2026.
- [7] C. Si, Y. Zhang, R. Li, Z. Yang, R. Liu, and D. Yang, "Design2code: Benchmarking multimodal code generation for automated front-end engineering," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2025, pp. 3956–3974.
- [8] J. Xiao, Y. Wan, Y. Huo, Z. Wang, X. Xu, W. Wang, Z. Xu, Y. Wang, and M. R. Lyu, "Interaction2code: Benchmarking mllm-based interactive webpage code generation from interactive prototyping," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE Press, 2025, p. 241–253. [Online]. Available: <https://doi.org/10.1109/ASE63991.2025.00028>
- [9] J. Xiao, M. Wang, M. H. Lam, Y. Wan, J. Liu, Y. Huo, and M. R. Lyu, "Designbench: A comprehensive benchmark for mllm-based front-end code generation," *arXiv preprint arXiv:2506.06251*, 2025.
- [10] Y. Wan, C. Wang, Y. Dong, W. Wang, S. Li, Y. Huo, and M. Lyu, "Divide-and-conquer: Generating ui code from screenshots," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 2099–2122, 2025.
- [11] L. Wei, Y. Liu, and S.-C. Cheung, "Taming android fragmentation: Characterizing and detecting compatibility issues for android apps," in *Proceedings of the 31st IEEE/ACM international conference on automated software engineering*, 2016, pp. 226–237.
- [12] H. Huang, C. Xu, M. Wen, Y. Liu, and S.-C. Cheung, "Conffix: Repairing configuration compatibility issues in android apps," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 514–525.
- [13] D. Zhao, Z. Xing, C. Chen, X. Xu, L. Zhu, G. Li, and J. Wang, "Seenomaly: Vision-based linting of gui animation effects against design-don't guidelines," in *Proceedings of the ACM/IEEE 42nd international conference on software engineering*, 2020, pp. 1286–1297.
- [14] Z. Liu, C. Chen, J. Wang, Y. Huang, J. Hu, and Q. Wang, "Owl eyes: Spotting ui display issues via visual understanding," in *Proceedings of the 35th IEEE/ACM international conference on automated software engineering*, 2020, pp. 398–409.
- [15] C. Chen, T. Su, G. Meng, Z. Xing, and Y. Liu, "From ui design image to gui skeleton: a neural machine translator to bootstrap mobile gui implementation," in *Proceedings of the 40th International Conference on Software Engineering*, 2018, pp. 665–676.
- [16] A. Mesbah and M. R. Prasad, "Automated cross-browser compatibility testing," in *Proceedings of the 33rd International Conference on Software Engineering*, 2011, pp. 561–570.
- [17] S. R. Choudhary, H. Versee, and A. Orso, "Webdiff: Automated identification of cross-browser issues in web applications," in *2010 IEEE International Conference on Software Maintenance*. IEEE, 2010, pp. 1–10.
- [18] S. R. Choudhary, M. R. Prasad, and A. Orso, "X-pert: Accurate identification of cross-browser issues in web applications," in *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 2013, pp. 702–711.
- [19] —, "Combining crawling and differencing to better detect cross-browser incompatibilities in web applications," in *Proceedings of the 5th IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2012, pp. 171–180.
- [20] T. Saar, M. Dumas, M. Kaljuve, and N. Semenenko, "Browserbite: cross-browser testing via image processing," *Software: Practice and Experience*, vol. 46, no. 11, pp. 1459–1477, 2016.
- [21] T. A. Walsh, G. M. Kapfhammer, and P. McMinn, "Automated layout failure detection for responsive web pages without an explicit oracle," in *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2017, pp. 192–202.
- [22] Y. Gui, Y. Wan, Z. Li, Z. Zhang, D. Chen, H. Zhang, Y. Su, B. Chen, X. Zhou, W. Jiang *et al.*, "Uicopilot: Automating ui synthesis via hierarchical code generation from webpage designs," in *Proceedings of the ACM on Web Conference 2025*, 2025, pp. 1846–1855.
- [23] S. Liang, N. Jiang, S. Qian, and L. Tan, "Waffle: Fine-tuning multimodal model for automated front-end development," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 24 786–24 802.
- [24] J. Xiao, Z. Zhang, Y. Wan, Y. Huo, Y. Liu, and M. R. Lyu, "Efficientuicoder: Efficient mllm-based ui code generation via input and output token compression," *arXiv preprint arXiv:2509.12159*, 2025.
- [25] J. Xiao, J. Qin, S. Li, M. H. Lam, Y. Wan, J.-t. Huang, Y. Huo, and M. R. Lyu, "Comuicoder: Component-based reusable ui code generation for complex websites via semantic segmentation and element-wise feedback," *arXiv preprint arXiv:2602.19276*, 2026.
- [26] Y. Wan, T. Liang, J. Xu, J. Xiao, Y. Huo, and M. R. Lyu, "From runnable to shippable: Multi-agent test-driven development for generating full-stack web applications from requirements," *arXiv preprint arXiv:2605.17242*, 2026.
- [27] BrowserStack, "Browserstack," <https://www.browserstack.com/>, 2026, accessed: 2026-06-29.
- [28] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [29] OpenAI, "Gpt-5.1: A smarter, more conversational chatgpt," <https://openai.com/index/gpt-5-1/>, 2025, accessed: 2026-06-29.
- [30] Google, "Gemini Canvas: Write, Code, and Create in One Space with AI," [Online]. Available: <https://gemini.google/overview/canvas/>, accessed: Jun. 18, 2026.
- [31] C. T. Le, T. O. Y. Siang, J. Xiao, Y. Wan, and Y. Huo, "Uibenchmark: A unified toolkit for design-to-code model evaluation," *arXiv preprint arXiv:2605.13141*, 2026.
- [32] StatCounter Global Stats, "Browser market share worldwide," <https://gs.statcounter.com/browser-market-share>, accessed: 2026-06-28.
- [33] —, "Desktop operating system market share worldwide," <https://gs.statcounter.com/os-market-share/desktop/worldwide>, accessed: 2026-06-28.
- [34] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, "Learning transferable visual models from natural language supervision," in *International conference on machine learning*. PmLR, 2021, pp. 8748–8763.
- [35] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE transactions on image processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [36] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, 2002, pp. 311–318.
- [37] C. Li, Y. Zheng, X. Huang, T. Fang, J. Xu, L. Chen, Y. Song, and H. Hu, "Webdevjudge: Evaluating (m) llms as critiques for web development quality," *arXiv preprint arXiv:2510.18560*, 2025.
- [38] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and psychological measurement*, vol. 20, no. 1, pp. 37–46, 1960.

- [39] W. M. Watanabe, D. A. Dos Santos, and C. De Oliveira, "Layout cross-browser failure classification for mobile responsive design web applications: Combining classification models using feature selection," *ACM Transactions on the Web*, vol. 17, no. 4, pp. 1–34, 2023.